BT4012 Kaggle Competition Report

Brian Lim Yen Sing (A0272973R)

1. Introduction

My final model ranks Bitcoin transactions by how likely they are to be illicit. It scored a public ROC AUC of 0.97357 and finished 3rd on the public leaderboard. Training covers time steps 1 to 35 of the Elliptic data (Weber et al., 2019) (called days below), where 31,235 transactions are labelled and 11.67% are illicit. The task is to score 15,329 labelled transactions from days 36–49.

Three facts shaped the approach:

- **The split is by time.** The model must work on later days it has never seen, so random cross-validation misleads.
- **Illicit cases are rare**, about one in nine, so the rare class needs deliberate weighting.
- **The data behave differently from day 23 onwards**, so offline validation rarely predicted the leaderboard.

The work went through three stages. First, I tuned gradient-boosted trees, reaching a ROC AUC of 0.95521 with XGBoost. Then, I explored TabPFN, a neural network pretrained on synthetic tables that scores new rows by reading labelled examples, with no training on this data. It achieved 0.96825 as a blend. The final improvements to 0.97357 came from blending TabPFN with a second such model, NVIDIA Kumo Tabular.

2. Methodology

2.1. Data preprocessing and feature engineering

The final model uses the 165 supplied features unchanged, with no augmentation. Through early experimentation, no feature change I tried gained on the leaderboard.

Cleaning was not needed. The features have no missing values. The only gaps are the labels of the 110,537 unlabelled training rows, which the final model does not use. There are no duplicate IDs or constant features. Scaling made no difference to trees, and the pretrained models get the raw values.

Train–test differences. The test days look very different from the training days: a classifier can tell training rows from test rows perfectly using the features alone. Six features (`feat_100`, `101`, `103`, `136`, `137`, `139`) mostly encode time, and for over 99% of test rows three features take values never seen in training. Dropping the six features lost 0.0042 on the leaderboard, and clipping test values into the training range barely changed the order of scores (Spearman 0.992–0.995), so I kept all features as supplied.

Graph features. `train.csv` holds every transaction of days 1–35, but `test.csv` contains only the labelled transactions of days 36–49. No training transaction has a neighbour without a feature row, while in test, on average 39% of a transaction's neighbours have none. Features built from neighbours therefore mean different things in train and test. This is the likely cause of the 0.054 loss in the table below, though I did not isolate which of the 22 features caused it. Transactions only link to others on the same day. So no test transaction touches a labelled training one, and you can't spread known labels across the graph to test.

Feature change (XGBoost)	Δ AUC, time folds	Δ public AUC
22 graph features: degree, two-hop counts, neighbour aggregates	+0.0435	**−0.0540**
Drop 21 distribution-shifted features	+0.0079	−0.0083
17 drifting features replaced by within-day ranks	+0.003 to +0.012	−0.0060
Counts of labelled neighbours	+0.0004 to +0.0025	0.0000

Using the 110,537 unlabelled rows through various methods such as PCA features and ten pseudo-labelling variants did not improve the leaderboard score either.

2.2. Model development and validation

Offline validation could not rank models reliably. I used time-ordered folds to tune XGBoost and only used the public leaderboard to judge.

Time-based folds. I used three folds. Each trains on all days up to a cut-off (day 15, 18 or 21) and is scored on the 14 days after it, so the model never sees the future. One XGBoost run can move by ±0.013 AUC from its random seed alone, so I averaged each setting over 5–10 seeds.

Data shift at day 23. Scored on days 30–35, a model trained on days 1–22 gets AUC 0.8739. Adding day 23 raises this to 0.9938, and day 24 alone gets 0.9922. A fold that scores 14 unseen days must stop training by day 21, so it never sees this shift. Folds after day 23 cannot separate models: eight very different models all scored 0.9959–0.9971. None of five validation setups ranked four known submissions in Kaggle's order, with the best rank correlation at +0.40.

Tuning. A 24-configuration random search picked depth 12, column sampling 0.7, learning rate 0.1 and 79 trees. Only the first two changes helped on the leaderboard, at about half their offline gain. Two near-identical files scored 0.0012 apart, so public gains under 0.001 are noise.

Class imbalance. For XGBoost, `scale_pos_weight` 7.57 (27,591 licit / 3,644 illicit) weighs both classes equally and `min_child_weight` 8 stops trees splitting on two or three illicit rows. Together they added +0.0052 on the leaderboard.

Pretrained models. TabPFN v3.5 (Hollmann et al., 2025; Prior Labs, 2026) and Kumo Tabular (NVIDIA, 2026a, 2026b) are tabular foundation models. Each reads a *context* of labelled rows and scores new rows in one forward pass, with no weights fitted to this data. A *bag* is one such pass: all illicit rows from a first day on, plus seeded random licit rows up to 8,000 or 12,000 rows, so an all-days context is 46% illicit and a day-18 one 28%. This is how the class imbalance is handled for these models. Bags differ in seed, first day and context size; their scores are turned into ranks and averaged, which suits AUC. The recipe, including first day, context sizes, partner model and weights was chosen on the public leaderboard, so it may overfit the public sample.

2.3. Model exploration and final strategy

The final model blends TabPFN with Kumo Tabular, the only one of five models I experimented with that improved the score when added to TabPFN.

Tree models. XGBoost was compared with seven other families: LightGBM, CatBoost, scikit-learn histogram boosting, random forest, extra trees, logistic regression and a tuned neural network (MLP). Only the MLP beat XGBoost offline and a 50/50 blend with it scored 0.94800 against 0.94812 without. A two-layer directed graph neural network scored 0.92745 alone. Blending XGBoost models trained from start days 1, 12, 18 and 23 gave the best tree result, 0.95672, within noise of the single model.

TabPFN. One bag alone scored 0.96026. On 3,000 rows from days 20–29 scored on days 30–35, it also beat XGBoost trained on the same rows (0.9978 against 0.9955). Bags whose context starts at day 18, a few days before the break, helped most; a day-23 start did no better. The best TabPFN blend (0.96825) has 11 bags, and every later TabPFN variant landed between 0.96402 and 0.96825.

Partner added to TabPFN	Spearman with TabPFN on test	Share	Public Δ AUC
XGBoost window blend (to an earlier 9-bag mix)	0.85	.15	−0.0005
CatBoost	0.73–0.75	.15	−0.0011
TabICLv2 (Qu et al., 2026)	0.92–0.93	.20	−0.0003
Other TabPFN v3 checkpoints	0.87–0.91	.25	−0.0012
NVIDIA Kumo Tabular	**0.86**	**.25**	**+0.0034**

Kumo's own seeds agree at Spearman 0.98, so its disagreement with TabPFN is consistent rather than random. CatBoost disagrees more but is much weaker (0.94800 alone), and the v3 checkpoints disagree almost as much and still lost. I never submitted Kumo alone, so its strength is inferred from one blend gain, the best of five partners on the same public sample.

Final recipe (submissions/blend_tabpfn_kumo22_55.csv): the TabPFN blend at .45 rank-averaged with a 22-run Kumo blend at .55. Shares are approximate, because each blend is ranked again before the final average.

Part	Runs	Context rows	First day	Share of final
TabPFN v3.5	6 + 1 bags	8,000 / 12,000	1	.11
TabPFN v3.5	3 + 1 bags	8,000 / 12,000	18	.34
Kumo Tabular large	10 + 1 runs	8,000 / all 14,380	18	.41
Kumo Tabular large	10 + 1 runs	12,000 / all 31,235	1	.14

My two final selections on Kaggle are this blend (0.97357) and the TabPFN blend alone (blend_tabpfn_recent75_12k.csv, 0.96825) as a hedge in case Kumo's gain on the public 30% does not hold on the private 70%.

3. Results and discussion

The final blend scored **0.97357** on the public leaderboard, 3rd place, +0.0184 over tuned XGBoost.

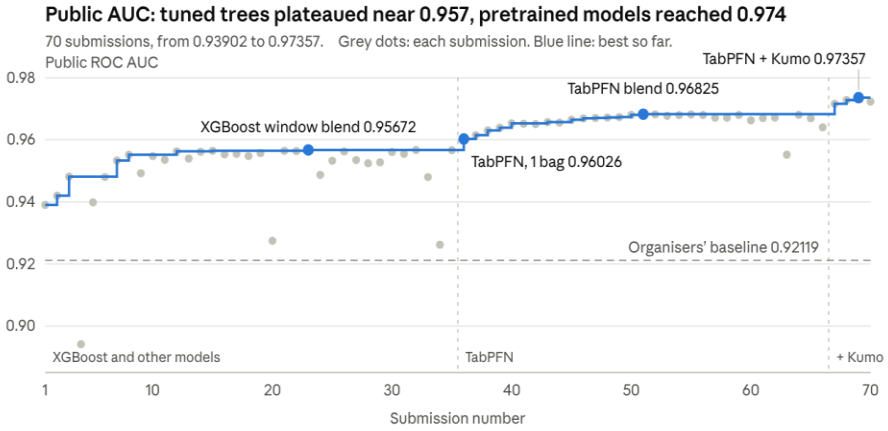


Figure 1: Public leaderboard scores of all 70 submissions

Tuning XGBoost settings gave the first +0.016 (0.93902 to 0.95521). The trees then plateaued while new features and other families failed. TabPFN and then Kumo each lifted the plateau, likely because they read recent labelled rows as context instead of fitting all history.

Kumo share in the final blend	Public AUC
.25 (12 Kumo runs)	0.97162
.33	0.97229
.40	0.97280
.55	**0.97357**

The score rose at each step (+0.0007, +0.0005, +0.0008), but each step is under the 0.001 noise level and all four share one public sample. So .55 is only the best of the four weights tried.

Late holdout. After fixing the recipe, I scored it on two held-out windows of training days. Neither favoured it over XGBoost, so the submission choice was entirely determined by the results of the public leaderboard.

Strengths

- No model weights are fitted to the training days. Adapting to a new period only requires choosing new context rows.
- Kumo's runs agree with each other (Spearman 0.98) yet rank test rows differently from TabPFN (0.86), so the blend gains where they disagree.

Limitations

- The public score rests on about 4,600 rows (30% of 15,329), only a few hundred of them illicit, so one standard deviation of a single score is about 0.01. Near-identical files differ by much less (about 0.001) because they share the same rows. The recipe was chosen on that sample, so the public score is optimistic and the private ranking may differ.
- No offline check supports the final model. On the late holdout it did not beat XGBoost.
- Kumo was never scored alone, and TabPFN reads only 8,000–12,000 context rows, not all 31,235 labelled ones.

4. Conclusion and reflection

Tuning gave the largest single step (+0.016, 0.939 to 0.955). After that only new model families gained: TabPFN +0.013 and Kumo +0.005, where features, more tuning and other tree models had stopped.

What worked. Pretrained tabular models used in context beat every tree model. Starting their context at day 18, near the day-23 break, helped TabPFN. Adding Kumo improved the score after further TabPFN variants had stopped helping.

What did not. Engineered graph features, dropped or re-ranked shifted features, unlabelled rows, post-processing and other model families all failed on the leaderboard, often after looking good offline. The supplied neighbour features (`feat_94`–`feat_165`) already carry much of the graph signal.

What I would do differently. Score each new model alone before blending it. Keep a holdout for blend weights, knowing one after the break saturates and can only catch a badly wrong weight. Test validation protocols against the leaderboard before trusting offline gains.

Implications for fraud detection. Illicit behaviour shifts over time, so a model fitted to one period can perform worse on the next without warning. No validation protocol here tracked that shift well. ROC AUC suits this task because investigators review transactions in ranked order. Models that adapt through their context looked well suited here, though I did not test them against retraining directly.

References

Code: github.com/brianxlim/bt4012-kaggle. `e1122376.ipynb` re-blends the committed per-run predictions into the submitted file exactly and runs the holdout above (seconds once its cache exists, about 10 minutes on a GPU before). With `E1122376_RERUN=1` it recomputed every run from the raw data in 2 hours on one GPU: all 22 Kumo runs matched the committed files exactly, and the final file agreed at Spearman 0.9996. `reports/progress.md` logs all 70 submissions.

Hollmann, N., Müller, S., Purucker, L., Krishnakumar, A., Körfer, M., Hoo, S. B., Schirrmeister, R. T., & Hutter, F. (2025). Accurate predictions on small data with a tabular foundation model. *Nature, 637*, 319–326. https://doi.org/10.1038/s41586-024-08328-6

NVIDIA. (2026a, September 29). *Kumo Tabular* [Blog post]. Hugging Face. https://huggingface.co/blog/nvidia/kumo-tabular

NVIDIA. (2026b). *Kumo-Tabular* (Version 1.0.1) [Model weights, OpenMDW 1.1 license]. Hugging Face. https://huggingface.co/nvidia/Kumo-Tabular/tree/v1.0.1

Prior Labs. (2026). *TabPFN-3.5* [Model card and weights, TABPFN-3.5 License v1.0]. Hugging Face. https://huggingface.co/Prior-Labs/tabpfn_3_5

Qu, J., et al. (2026). TabICLv2: A better, faster, scalable, and open tabular foundation model. In *Proceedings of the 43rd International Conference on Machine Learning*. https://arxiv.org/abs/2602.11139

Weber, M., Domeniconi, G., Chen, J., Weidele, D. K. I., Bellei, C., Robinson, T., & Leiserson, C. E. (2019). *Anti-money laundering in bitcoin: Experimenting with graph convolutional networks for financial forensics* [Paper presentation]. KDD '19 Workshop on Anomaly Detection in Finance. https://doi.org/10.48550/arXiv.1908.02591